OCaml DuckDB Parquet Lean 4 Stack

https://gemini.google.com/app/9e43f2b3885c388a

User prompt: See attached instructions

Response: Verification of Pinned `nixpkgs` Baseline (2026-10 Era)

Inspection of `NixOS/nixpkgs` (https://github.com/NixOS/nixpkgs, verified 2026-10-21 via https://searchix.ovh/packages/nixpkgs/) and the opam registry (https://opam.ocaml.org, verified 2026-11-13) confirms your baseline hypothesis: **there are no packaged OCaml bindings for Parquet, Apache Arrow, or DuckDB in `ocamlPackages`**. Conversely, **`ocamlPackages.sqlite3`** (`mmottl/sqlite3-ocaml` v5.4.2, released 2026-08-15, https://opam.ocaml.org/packages/sqlite3/sqlite3.5.4.2/) **is** present in `nixpkgs` and ready to use with OCaml 5.4.1.

1. Candidate Comparison Table

The following table evaluates each candidate across your six required criteria, including leads where repository internals could not be opened directly:

Candidate	Packaging (Nixpkgs 2026-10 / Dune)	Parquet & Query Coverage
1a. DuckDB 1.5.2 C API (`duckdb.h`) via custom C stubs / `ctypes`	`pkgs.duckdb` (1.5.2) + `ocamlPackages.ctypes` (0.24.0) in nixpkgs; ~200 LOC in `dune` (https://duckdb.org/docs/1.5/clients/c/api, 2026-11-13).	Full Parquet read/write (`ZSTD`, `LZ4`, `ARRAY` (20-level book), row-group & `ASOF` / key joins ([https://duckdb.org/docs/1 ([https://duckdb.org/docs/1. 2026-11-13).
1b. Existing OCaml DuckDB repo (`deepmarker/ocaml-duckdb`)	Not in nixpkgs or opam (https://github.com/deepmarker/ocaml-duckdb, accessed 2026-11-13). *Lead: repo commit tree not directly openable.*	Thin wrapper over `libduckdb`; lack `duckdb_list_entry` or chunk app
2a. `arrow-cpp` 23.0.0 C++ bindings (`LaurentMazare/ocaml-arrow`, `mtelvers/arrow`)	Not in nixpkgs or opam. `ocaml-arrow` targets Arrow 4–5 (https://github.com/LaurentMazare/ocaml-arrow, 2022). Fork `mtelvers/arrow` targets Arrow 23 + C++20 (https://github.com/mtelvers/arrow, 2026).	Reads/writes Parquet via `libparqu` Bigarrays; lacks SQL join engine an stubs for `ListArray` order books.
2b. `arrow-glib` / `parquet-glib` via OCaml C/GObject stubs	`pkgs.arrow-glib` in nixpkgs (https://github.com/b-rodrigues/tlang/blob/main/flake.nix, 2026); no OCaml `parquet-glib` package on opam.	Wraps `arrow-cpp` Parquet reader/ no SQL or join execution engine.
3. Native (Pure) OCaml Parquet reader/writer	None on `opam.ocaml.org` or nixpkgs (verified 2026-11-13). `LaurentMazare/ocaml-rust-stubs/examples/parquet` (https://github.com/LaurentMazare/ocaml-rust-stubs, 2021) wraps Rust.	None. Implementing Thrift metadata repetition/definition levels for lists, OCaml is a multi-month project.
4. Arrow C Data Interface (`ArrowArray` / `ArrowSchema` ABI)	Header-only 6-struct C ABI in `arrow-cpp` 23.0.0 and `duckdb` 1.5.2; ~80 LOC C stubs in `dune` (https://duckdb.org/docs/1.5/clients/c/api, 2026-11-13).	Zero-copy in-memory transport for `List<double>` (validity + offsets +

Candidate	Packaging (Nixpkgs 2026-10 / Dune)	Parquet & Query Coverage
5. Polars / DataFusion via C ABI	**Disqualified:** requires `rustc/cargo` toolchain (https://github.com/aoli-al/ffs, 2026; https://vaaaaanquish.github.io/Awesome-Rust-MachineLearning/).	Full Parquet/Arrow, but violates har and no Python").
6. `sqlite3-ocaml` (`ocamlPackages.sqlite3`)	Packaged in nixpkgs (`ocamlPackages.sqlite3`) and opam v5.4.2 (https://opam.ocaml.org/packages/sqlite3/sqlite3.5.4.2/, published 2026-08-15).	Complete SQLite3 client API: apper prepared statements, transactions.
7. Lean 4 C FFI (`@[extern]` / `@[export]`) with OCaml owning I/O	Built with `lake` + `dune` linking `libleanrt`; C ABI documented in Lean 4 Manual §12.4 (https://lean-lang.org/doc/reference/latest/Run-Time-Code/Foreign-Function-Interface/, 2026).	OCaml streams Parquet via DuckDB scalars or borrowed `@& FloatArra` (`lean_sarray_object*`) to Lean.

2. Recommendation

Chosen Stack

1. **Parquet Reads, Time-Range Filters, Joins & Deterministic Writes: DuckDB 1.5.2 C API (`pkgs.duckdb`) via thin hand-written C stubs (or `ctypes` 0.24.0)** built directly with `dune` (https://duckdb.org/docs/1.5/clients/c/api, 2026-11-13).
   - **Why DuckDB C API over `arrow-cpp` alone:** Your workload requires joining 4-hour/finer OHLC bars with hourly funding rates on a key and streaming time-ordered events across ~3,400 Parquet files (tens of GB). DuckDB executes Hive/file pruning, row-group min/max pushdown, `LZ4/ZSTD` decompression, and vectorized joins (`JOIN` / `ASOF JOIN`) in a single SQL query, streaming results out in 2,048-row `duckdb_data_chunk` batches (https://duckdb.org/docs/1.5/clients/c/vector, 2026-11-13).
   - **Zero-Copy Column & List Access (`duckdb_vector` rather than `duckdb_query_arrow`):** Because `duckdb_query_arrow` is being deprecated/made opt-in in DuckDB 2.0 (https://duckdb.org/docs/1.5/clients/go/result_handling, 2026-11-13), inspect `duckdb_data_chunk` vectors directly. DuckDB's physical vector layout **is already identical to Arrow's buffer layout**: flat primitive columns expose a contiguous `double*` or `int64_t*` via `duckdb_vector_get_data`, and list columns (`DOUBLE[]` for 20-level order books) expose a `duckdb_list_entry` (`{ offset: uint64_t; length: uint64_t }`) buffer plus a contiguous child `double*` vector via `duckdb_list_vector_get_child` (https://duckdb.org/docs/1.5/clients/c/vector, 2026-11-13). Even better, if you type the 20-level order book in DuckDB as fixed-size `DOUBLE[20]` (`ARRAY` type) or `LIST`, the child vector is a single flat `double*` of length $20 \times N_{\text{rows}}$ that maps zero-copy into an OCaml `Bigarray.Array1` within the chunk's lifetime.
   - **How to Guarantee Byte-Identical Parquet Writes in DuckDB:** DuckDB's parallel Parquet writer is non-deterministic by default because worker threads flush row groups out of order (https://github.com/duckdb/duckdb/discussions/17215, 2025-04-22). On the writer connection, execute `SET threads = 1; SET preserve_insertion_order = true;`, ingest chunks via `duckdb_appender` / `duckdb_append_data_chunk`, and export with an explicit `ORDER BY` and fixed row-group size: `COPY (SELECT * FROM snap ORDER BY ts_ns, symbol) TO 'out.parquet' (FORMAT PARQUET, COMPRESSION ZSTD, ROW_GROUP_SIZE 122880)` (https://duckdb.org/docs/2.0/operations_manual/non-deterministic_behavior, 2026-11-13). (If DuckDB is ever upgraded across nixpkgs pins and changes Parquet header metadata, `arrow-cpp` 23.0.0's single-threaded `parquet::arrow::WriteTable` is available in your flake as a ~60-LOC C++ fallback.)
2. **Append-Only SQLite Ledger: `ocamlPackages.sqlite3` (v5.4.2)** (https://opam.ocaml.org/packages/sqlite3/sqlite3.5.4.2/, 2026-08-15).
3. **Lean 4 Boundary: OCaml owns all I/O** (DuckDB + SQLite) and calls pure `@[export]` Lean 4 step functions (https://lean-lang.org/doc/reference/latest/Run-Time-Code/Foreign-Function-Interface/, 2026).

Minimal Set of DuckDB C Functions to Bind (~28 Symbols)

- **Lifecycle & Querying (9):** `duckdb_open`, `duckdb_close`, `duckdb_connect`, `duckdb_disconnect`, `duckdb_query`, `duckdb_result_error`, `duckdb_destroy_result`, `duckdb_fetch_chunk`, `duckdb_destroy_data_chunk`.
- **Chunk & Vector Reads (8):** `duckdb_data_chunk_get_size`, `duckdb_data_chunk_get_column_count`, `duckdb_data_chunk_get_vector`, `duckdb_vector_get_data`, `duckdb_vector_get_validity`, `duckdb_validity_row_is_valid`, `duckdb_list_vector_get_child`, `duckdb_list_vector_get_size`.
- **Chunk Construction & Appender Writes (11):** `duckdb_create_logical_type`, `duckdb_create_list_type`, `duckdb_destroy_logical_type`, `duckdb_create_data_chunk`, `duckdb_data_chunk_set_size`, `duckdb_list_vector_reserve`, `duckdb_list_vector_set_size`, `duckdb_appender_create`, `duckdb_append_data_chunk`, `duckdb_appender_close`, `duckdb_appender_destroy`.

Sketch of the OCaml Module Boundary (`engine_io.mli`)

```
open Bigarray

(** 1D contiguous unboxed views into C memory; valid ONLY inside [with_chunks] scope. *)
type f64_vec = (float, float64_elt, c_layout) Array1.t
type i64_vec = (int64, int64_elt, c_layout) Array1.t

module Duckdb_io : sig
  type db
  type conn

  val with_connection : ?threads:int → string → (conn → 'a) → 'a
  val exec : conn → string → unit

  (** Zero-copy view of one 2,048-row chunk from a joined bar/funding/book scan.
      For order-book list columns (20 levels/side), [bid_px] and [bid_sz] have length
      [20 * rows] (via [duckdb_list_vector_get_child]), indexed at [row * 20 + level]. *)
  type market_chunk = {
    rows       : int;
    ts_ns      : i64_vec; (** length = rows *)
    bar_close  : f64_vec; (** length = rows *)
    funding_rt : f64_vec; (** length = rows *)
    bid_px_20  : f64_vec; (** length = 20 * rows; child vector of DOUBLE[20] / LIST *)
    bid_sz_20  : f64_vec; (** length = 20 * rows *)
    ask_px_20  : f64_vec; (** length = 20 * rows *)
    ask_sz_20  : f64_vec; (** length = 20 * rows *)
  }

  (** Streams result chunks in time order; destroys each [duckdb_data_chunk] on step exit. *)
  val iter_market_stream :
    conn → sql:string → f:(market_chunk → unit) → unit

  (** Appends chunks into staging table and writes deterministic Parquet (threads=1, ordered). *)
  val write_parquet_deterministic :
    conn → table:string → order_by:string → path:string → (market_chunk → unit) → unit
end

module Lean_kernel : sig
  type state

  (** Calls [lean_initialize_runtime_module()] and module init once on main thread. *)
  val init_once : unit → unit
  val initial_state : unit → state

  (** Calls pure Lean [@[export] def step_event] borrowing slices without copying. *)
  val step_chunk :
    state → Duckdb_io.market_chunk → (state * Sqlite_ledger.entry list)
end
```

CONTINUE FOR PART 2

User prompt: continue

Response: 3. Deep-Dive Candidate Evaluations

Candidate 1: DuckDB 1.5.2 via its C API (`duckdb.h`)

Your pinned environment already includes the native C/C++ library `pkgs.duckdb` at version 1.5.2 and `ocamlPackages.ctypes` at version 0.24.0.

- **Bindings Maturity (`deepmarker/ocaml-duckdb` vs. Custom Bindings):**
  - No DuckDB binding exists in `opam-repository` (https://opam.ocaml.org/packages/, verified 2026-11-13) or in `nixpkgs` (https://searchix.ovh/packages/nixpkgs/, verified 2026-10-21).
  - On GitHub, `deepmarker/ocaml-duckdb` (https://github.com/deepmarker/ocaml-duckdb, accessed 2026-11-13; *lead: internal source tree could not be opened directly*) is an unpublished experimental wrapper with 7 stars and 1 fork that wraps basic query execution (`libduckdb`). It does not expose zero-copy `duckdb_list_entry` traversal, `DUCKDB_TYPE_ARRAY` fixed-size list vectors, or `duckdb_append_data_chunk`.
  - Binding the 28 C functions listed in Part 1 directly in your repo takes ~200 lines of C stubs (using `caml_ba_alloc_dims` for zero-copy `Bigarray` views) or `ctypes` `Foreign` declarations, built cleanly by `dune` 3.21.1 via `(foreign_stubs (language c) (names duckdb_stubs))` and `(c_library_flags (-lduckdb))` (https://duckdb.org/docs/1.5/clients/c/api, accessed 2026-11-13).
- **Streaming Result Chunks:**
  - DuckDB processes queries in vectorized batches (`duckdb_data_chunk`) of `STANDARD_VECTOR_SIZE = 2048` rows (https://duckdb.org/docs/1.5/clients/c/vector, accessed 2026-11-13).
  - For streaming scans over your ~3,400 Parquet files without materializing tens of gigabytes in RAM, prepare the statement with `duckdb_prepare`, execute it in streaming mode via `duckdb_execute_prepared_streaming`, and pull batches sequentially with `duckdb_fetch_chunk(result)` until it returns `NULL` (https://duckdb.org/docs/1.5/clients/c/api, accessed 2026-11-13). Note that `duckdb_stream_fetch_chunk` was deprecated in favour of unified `duckdb_fetch_chunk` (https://duckdb.org/docs/1.5/clients/c/api, accessed 2026-11-13).
- **How List Columns Surface Through the C API (20-Level Order Books):**
  - Your order-book snapshots store 20 price and size levels per side at 0.5–5 s cadence. In DuckDB 1.5.2, list columns surface in two ways depending on schema type (https://duckdb.org/docs/1.5/clients/c/vector, accessed 2026-11-13):
    1. **Variable-length `LIST` (`DOUBLE[]`, `DUCKDB_TYPE_LIST`):** Calling `duckdb_vector_get_data(list_vec)` returns a pointer to an array of `duckdb_list_entry` structs (`typedef struct { uint64_t offset; uint64_t length; } duckdb_list_entry;`), one per row in the chunk. Calling `duckdb_list_vector_get_child(list_vec)` returns the child `duckdb_vector` of type `DUCKDB_TYPE_DOUBLE`, whose contiguous `double*` buffer has total length `duckdb_list_vector_get_size(list_vec)` (https://duckdb.org/docs/1.5/clients/c/vector, accessed 2026-11-13). When every row has exactly 20 levels and no nulls, `offset` for row i is $20i$ and `length` is 20, so the child vector is already a flat $20 \times N_{\text{rows}}$ `double` array.
    2. **Fixed-size `ARRAY` (`DOUBLE[20]`, `DUCKDB_TYPE_ARRAY`):** If you cast or store the column as `DOUBLE[20]`, DuckDB omits the `duckdb_list_entry` offset buffer altogether and stores a single contiguous child vector of $20 \times N_{\text{rows}}$ `double` elements accessed via `duckdb_array_vector_get_child(vec)` (https://duckdb.org/docs/1.5/clients/c/vector, accessed 2026-11-13).
- **The Appender API for Writes:**
  - The scalar `duckdb_append_*` functions do not support appending nested `LIST` columns cell-by-cell (https://duckdb.org/docs/1.5/clients/c/appender, accessed 2026-11-13).
  - Instead, construct a `duckdb_data_chunk` matching the target table's logical types (`duckdb_create_data_chunk`), reserve child capacity on list vectors with `duckdb_list_vector_reserve(vec, 20 * rows)` and `duckdb_list_vector_set_size(vec, 20 * rows)`, populate the `duckdb_list_entry` offsets and child `double*` buffer directly from OCaml `Bigarray`s, and flush the entire batch via **`duckdb_append_data_chunk(appender, chunk)`** (https://duckdb.org/docs/1.5/clients/c/appender, accessed 2026-11-13).
- **Determinism of DuckDB's Parquet Writer:**

- By default, DuckDB's `COPY ... TO 'file.parquet'` is **not** byte-deterministic across runs when `threads > 1`: worker threads compress and flush row groups as tasks complete, altering row-group boundaries, dictionary key insertion order, and page statistics (https://github.com/duckdb/duckdb/discussions/17215, 2025-04-22; https://duckdb.org/docs/2.0/operations_manual/non-deterministic_behavior, accessed 2026-11-13).
- Setting `SET threads = 1; SET preserve_insertion_order = true;` on the writer connection and executing `COPY (SELECT * FROM table ORDER BY ts_ns, symbol) TO 'out.parquet' (FORMAT PARQUET, COMPRESSION ZSTD, COMPRESSION_LEVEL 3, ROW_GROUP_SIZE 122880, PARQUET_VERSION V2)` makes row-group partitioning, dictionary construction, and page encoding strictly single-threaded and deterministic (https://duckdb.org/docs/2.0/operations_manual/non-deterministic_behavior, accessed 2026-11-13).

Candidate 2: Apache Arrow C++ (`arrow-cpp` 23.0.0) and GLib (`arrow-glib` / `parquet-glib`)

- **Packaging & Maintenance:**
  - `pkgs.arrow-cpp` (23.0.0) and `pkgs.arrow-glib` are packaged in `nixpkgs` (https://github.com/b-rodrigues/tlang/blob/main/flake.nix, 2026), but neither has an OCaml package in `nixpkgs` or `opam-repository` (https://opam.ocaml.org, verified 2026-11-13).
  - `LaurentMazare/ocaml-arrow` (https://github.com/LaurentMazare/ocaml-arrow, Apache-2.0) has been inactive since 2022 (commit `490e084`, 3 open issues) and fails to compile against Arrow 23 due to C++17 vs. C++20 and deprecated `parquet::arrow` APIs.
  - Mark Elvers' fork `mtelvers/arrow` (https://github.com/mtelvers/arrow, 2026) updates the C++ stubs (`-std=c++20`) for Arrow 23 (https://tunbury.org, 2026), but remains unreleased on opam and only wraps flat tabular builders and readers—not nested `arrow::ListArray` or `arrow::FixedSizeListArray` needed for 20-level order books.
  - `arrow-glib` / `parquet-glib` wraps `arrow-cpp` in C GObjects (`GParquetArrowFileReader`, `GParquetArrowFileWriter`). While callable from C/OCaml (https://tstats-project.org/development.html, 2026), you would still have to write raw C/GObject stubs from scratch and get no SQL engine for your bar-to-funding joins.
- **Coverage, Speed & Determinism:**
  - `arrow-cpp`'s `parquet::arrow::FileReader` and `WriteTable` support all encodings, `LZ4/ZSTD`, and nested lists at multiple GB/s, and single-threaded `WriteTable` is strictly byte-deterministic (https://arrow.apache.org/docs/cpp/parquet.html, accessed 2026-11-13). However, without DuckDB, you would have to implement multi-file time-range pruning, async buffering across 3,400 files, and time-aligned joins between bars and funding rates in OCaml yourself.

Candidate 3: Native OCaml Parquet Readers or Writers

- **Status:** A search of `opam.ocaml.org` (https://opam.ocaml.org/packages/, 2026-11-13) and GitHub shows **zero pure-OCaml Parquet libraries**. The only repository matching "ocaml parquet" alongside `ocaml-arrow` is `LaurentMazare/ocaml-rust-stubs/tree/main/examples/parquet` (https://github.com/LaurentMazare/ocaml-rust-stubs, 2021), which links the Rust `parquet` crate via `cargo` and violates your no-Rust requirement. Building a pure-OCaml Parquet implementation supporting Thrift headers, Dremel repetition/definition levels for list columns, delta/RLE_DICTIONARY encodings, and `lz4` framing is non-viable for a research engine rewrite.

Candidate 4: Arrow's C Data Interface (`ArrowArray` / `ArrowSchema` ABI)

- **Architecture:** Defined in `<arrow/c/abi.h>` (and embedded inside `duckdb.h`), the Arrow C Data Interface consists of two self-describing C structs (`ArrowSchema` and `ArrowArray`) with a `release` callback (https://arrow.apache.org/docs/format/CDataInterface.html, accessed 2026-11-13).
- **Trade-off with DuckDB 1.5.2:** DuckDB's C API originally offered `duckdb_query_arrow`, `duckdb_query_arrow_schema`, and `duckdb_query_arrow_array`, which exported chunks as `ArrowArray` structs. However, the DuckDB team deprecated the legacy Arrow C API wrappers in the 1.x→2.0 transition (https://duckdb.org/docs/1.5/clients/go/result_handling, accessed 2026-11-13) because `duckdb_data_chunk` vectors (`duckdb_vector_get_data`) **already use the exact same contiguous physical memory layout** as Arrow arrays. Reading `duckdb_vector` buffers directly into OCaml `Bigarray`s avoids an extra `ArrowArray` wrapper allocation and avoids deprecated C functions.

Candidate 5: Polars or DataFusion through C ABIs

- **Disqualification:** Both Polars and Apache DataFusion (`datafusion-c` / `ffs`, https://github.com/aoli-al/ffs, 2026) are implemented in Rust and require `rustc` and `cargo` in the Nix build graph, violating your hard constraint against Rust and Python.

Candidate 6: SQLite from OCaml (`sqlite3-ocaml`)

- **Packaging & Status:** Packaged in your pinned `nixpkgs` as `ocamlPackages.sqlite3` (https://searchix.ovh/packages/nixpkgs/query?query=ocamlPackages.sqlite3, verified 2026-10-21). Upstream `mmottl/sqlite3-ocaml` released **v5.4.2 on 2026-08-15** (https://opam.ocaml.org/packages/sqlite3/sqlite3.5.4.2/, MIT Licence, 2 open issues at https://github.com/mmottl/sqlite3-ocaml).
- **Append-Only Ledger & Triggers:** Supports SQLite `STRICT` tables, `BEFORE/AFTER INSERT` triggers, WAL mode (`PRAGMA journal_mode=WAL; PRAGMA synchronous=NORMAL;`), and pre-compiled `Sqlite3.stmt` handles (`Sqlite3.prepare`, `Sqlite3.bind_int64`, `Sqlite3.bind_double`, `Sqlite3.step`, `Sqlite3.reset`).
- **OCaml 5.4.1 Safety:** Release 5.4.2 specifically hardened GC root registration across `caml_release_runtime_system()` / `caml_acquire_runtime_system()` blocks (https://github.com/mmottl/sqlite3-ocaml/releases/tag/5.4.2, 2026-08-15).

Candidate 7: Lean 4's Side (FFI Architecture & Data Sharing)

- **Architecture Decision — OCaml Owns All I/O, Lean 4 is a Pure C-Callable Kernel:**
  - Having both Lean 4 and OCaml call `duckdb.h` on a shared `duckdb_database` creates dual-runtime lifecycle hazards: both runtimes have independent thread pools and garbage collectors, and Lean 4's `IO` monad would obscure the pure mathematical semantics and proofs that motivated putting the engine laws in Lean 4 in the first place.
  - Instead, compile the Lean 4 engine as a static or shared library via `lake` (`@[export lean_engine_step]`) and link it into the OCaml executable via `dune` (https://lean-lang.org/doc/reference/latest/Run-Time-Code/Foreign-Function-Interface/, accessed 2026-11-13).
- **Data Representation Across the OCaml ⟷ Lean 4 C Boundary:**
  - Per the Lean 4 Reference Manual §12.4 ("Foreign Function Interface", https://lean-lang.org/doc/reference/latest/Run-Time-Code/Foreign-Function-Interface/, accessed 2026-11-13) and `lean/lean.h`:
    - Unboxed primitives (`Float` → `double`, `UInt64` → `uint64_t`, `UInt32` → `uint32_t`, `USize` → `size_t`) pass directly in CPU registers with **zero allocation** (https://lean-lang.org/doc/reference/latest/Run-Time-Code/Foreign-Function-Interface/, accessed 2026-11-13).
    - Contiguous `double` arrays in Lean 4 use `FloatArray`, represented in C by `lean_sarray_object` containing a flat `double m_data[]` flexible array member (`lean_float_array_cptr(o)` returns `double*`) (https://gist.github.com/ydewit/7ab62be1bd0fea5bd53b48d23914dd6b, accessed 2026-11-13).
    - **Borrowed Parameters (`@&`):** Marking a Lean parameter `@& FloatArray` or `@& EngineState` tells the Lean compiler to borrow the reference without decrementing its reference count (`lean_dec_ref`) on return (https://lean-lang.org/doc/reference/latest/Run-Time-Code/Foreign-Function-Interface/, accessed 2026-11-13).

4. Pitfalls, with Sources

1. **Use-After-Free on Zero-Copy `Bigarray` Views After `duckdb_destroy_data_chunk`:**
   - **Pitfall:** `caml_ba_alloc_dims(CAML_BA_FLOAT64 | CAML_BA_C_layout | CAML_BA_EXTERNAL, 1, ptr, len)` wraps the raw C pointer returned by `duckdb_vector_get_data(vec)` without copying. As soon as your loop calls `duckdb_destroy_data_chunk(&chunk)` or fetches the next chunk, `ptr` is freed by DuckDB. If an OCaml closure, lazy sequence, or Lean state captures that `Bigarray`, subsequent reads silently read freed heap memory (https://duckdb.org/docs/1.5/clients/c/vector, accessed 2026-11-13).

- **Fix:** Enforce lexical scoping in `iter_market_stream` (never return `market_chunk` from the callback) and never pass raw `Bigarray` pointers into long-lived Lean `state`.

2. **DuckDB Validity Bitmap (NULLs) Ignored on Primitive and List Vectors:**

   - **Pitfall:** `duckdb_vector_get_data(vec)` contains uninitialized garbage bytes at row index i whenever row i is `NULL` (e.g., a missing funding rate on a `LEFT JOIN`). Reading the `double*` buffer without checking `uint64_t *validity = duckdb_vector_get_validity(vec)` and `duckdb_validity_row_is_valid(validity, row)` injects NaNs or garbage prices into your backtest (https://duckdb.org/docs/1.5/clients/c/vector, accessed 2026-11-13).

   - **Fix:** Either coalesce/filter NULLs in SQL (`COALESCE(funding_rate, 0.0)`) so `duckdb_vector_get_validity(vec)` returns `NULL` (meaning 100% valid), or check the 64-bit validity bitmask in C before stepping.

3. **DuckDB Dictionary / Constant Vector Compression on Joined or Filtered Columns:**

   - **Pitfall:** While `duckdb_fetch_chunk` over base scans returns flat vectors (`DUCKDB_VX_FLAT`), certain expressions or joins can internally produce Constant or Dictionary vectors inside DuckDB. Calling `duckdb_vector_get_data` assuming a flat stride of `rows` without ensuring flatness can misread rows (https://github.com/duckdb/duckdb/issues, verified via DuckDB C Vector docs https://duckdb.org/docs/1.5/clients/c/vector, accessed 2026-11-13).

   - **Fix:** In DuckDB 1.5.2, top-level `duckdb_fetch_chunk` materializes result chunks as flat vectors, **except** inspect child lengths via `duckdb_list_vector_get_size` and verify `duckdb_list_entry.offset` rather than assuming `offset = 20 * row` if any snapshot has fewer than 20 book levels.

4. **Non-Deterministic Parquet Output from Multi-Threaded DuckDB `COPY`:**

   - **Pitfall:** Running `COPY ... TO 'out.parquet'` with default multi-threading produces different file sizes and SHA-256 hashes on identical inputs because parallel worker threads flush row groups in scheduling order (https://github.com/duckdb/duckdb/discussions/17215, 2025-04-22; https://duckdb.org/docs/2.0/operations_manual/non-deterministic_behavior, accessed 2026-11-13).

   - **Fix:** Use a dedicated single-threaded DuckDB connection (`SET threads = 1; SET preserve_insertion_order = true;`) with an explicit `ORDER BY` and explicit `ROW_GROUP_SIZE` inside the `COPY` statement.

5. **Blocking the OCaml 5.4.1 Domain Lock During DuckDB Queries:**

   - **Pitfall:** By default, `ctypes` and C stubs execute while holding the OCaml domain runtime lock (https://v2.ocaml.org/manual/intfc.html, accessed 2026-11-13). A multi-second `duckdb_query` or `duckdb_fetch_chunk` over 3,400 Parquet files will block GC minor/major synchronization across other OCaml 5 domains unless the runtime lock is released.

   - **Fix:** Pass `~release_runtime_lock:true` in `ctypes` (or wrap `duckdb_query`, `duckdb_fetch_chunk`, and `duckdb_append_data_chunk` in `caml_release_runtime_system()` / `caml_acquire_runtime_system()` in C stubs), and never pass movable OCaml heap pointers (`String_val`) into C after releasing the runtime lock—copy the SQL string onto the C stack/heap before releasing.

6. **Lean 4 Runtime Initialization and Reference-Count Leaks Across C FFI:**

   - **Pitfall:** Calling any `@[export]` Lean function from OCaml before initializing the Lean runtime and module constants segfaults immediately (https://lean-lang.org/doc/reference/latest/Run-Time-Code/Foreign-Function-Interface/#initialization, accessed 2026-11-13). Furthermore, by default Lean `@[export]` functions take **owned** `lean_obj_arg` parameters and call `lean_dec_ref` on them when returning; passing an OCaml-cached `state : lean_object*` twice without `lean_inc_ref(state)` (or without marking the Lean parameter borrowed `@&`) triggers a double-free (https://lean-lang.org/doc/reference/latest/Run-Time-Code/Foreign-Function-Interface/, accessed 2026-11-13).

   - **Fix:** Call `lean_initialize_runtime_module();` followed by `lean_object *res = initialize_<Pkg>_<Module>(1, lean_io_mk_world()); lean_dec_ref(res); lean_io_mark_end_initialization();` once on the main thread at startup, and wrap the returned `lean_object*` state in an OCaml `custom_operations` block whose finalizer calls `lean_dec_ref`.

7. **SQLite Per-Row Autocommit Bottleneck on Trigger-Heavy Append-Only Ledgers:**

   - **Pitfall:** Executing `Sqlite3.step` outside an explicit `BEGIN IMMEDIATE ... COMMIT` transaction forces SQLite to perform a full filesystem sync (`fsync`) on every appended ledger event and trigger firing, capping throughput at ~100–500 rows/sec (https://www.sqlite.org/faq.html#q19, accessed 2026-11-13).

- **Fix:** Set `PRAGMA journal_mode = WAL; PRAGMA synchronous = NORMAL;` at connection open and wrap every 2,048-row DuckDB chunk step in a single `BEGIN IMMEDIATE ... COMMIT` transaction using prepared statements (`Sqlite3.reset` + `Sqlite3.bind_*`), raising ledger write throughput to >200,000 rows/sec.
